

Cross-Validation in Python

Implementation Strategies using Scikit-Learn

```
from sklearn.model_selection import ...
```

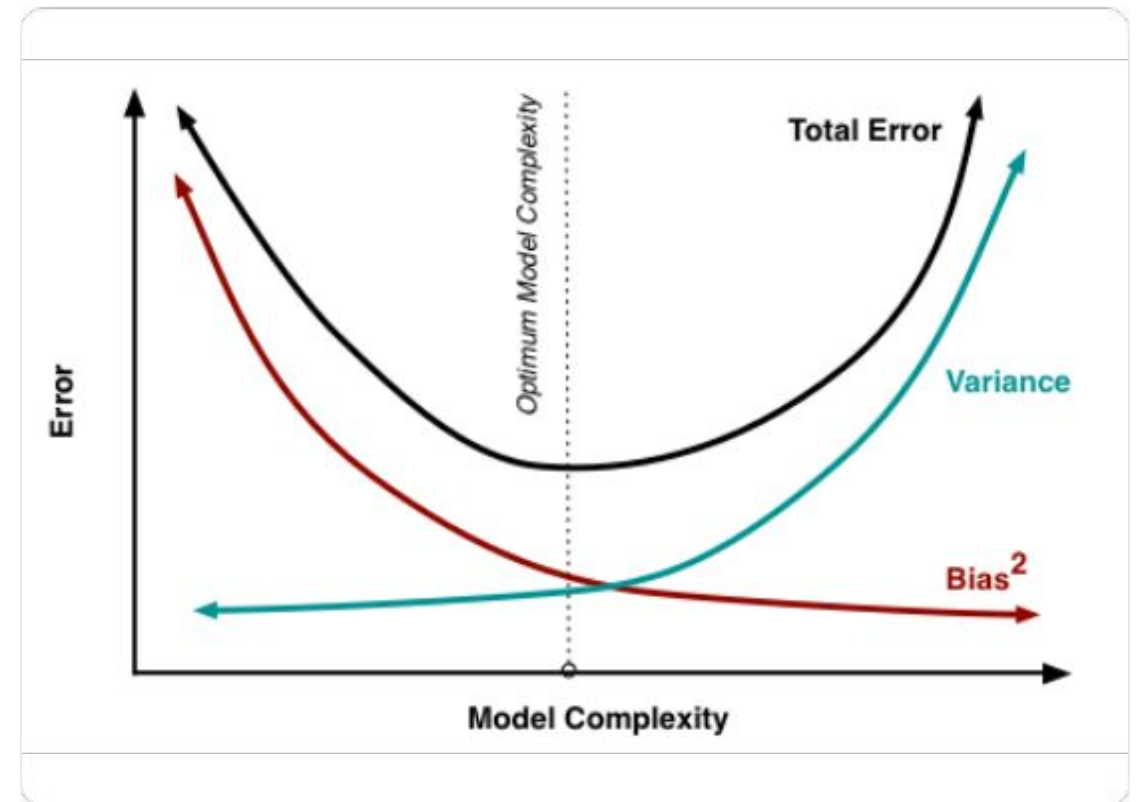
Why Cross-Validation?

The Problem with Single Splits

A single train/test split (e.g., 80/20) is high variance. The test score depends heavily on *which* data points end up in the test set.

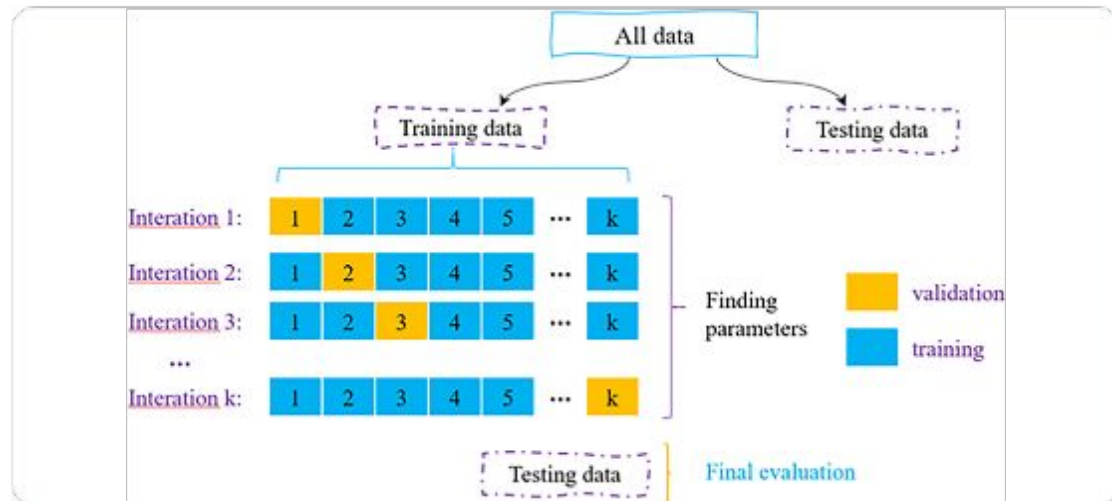
The Solution

- Train multiple models on different subsets.
- Average the scores to get a stable estimate.
- Ensures every data point is used for testing exactly once (in K-Fold).



1. Standard K-Fold

Splits data into **K** equal parts (folds). Train on K-1, Test on 1.



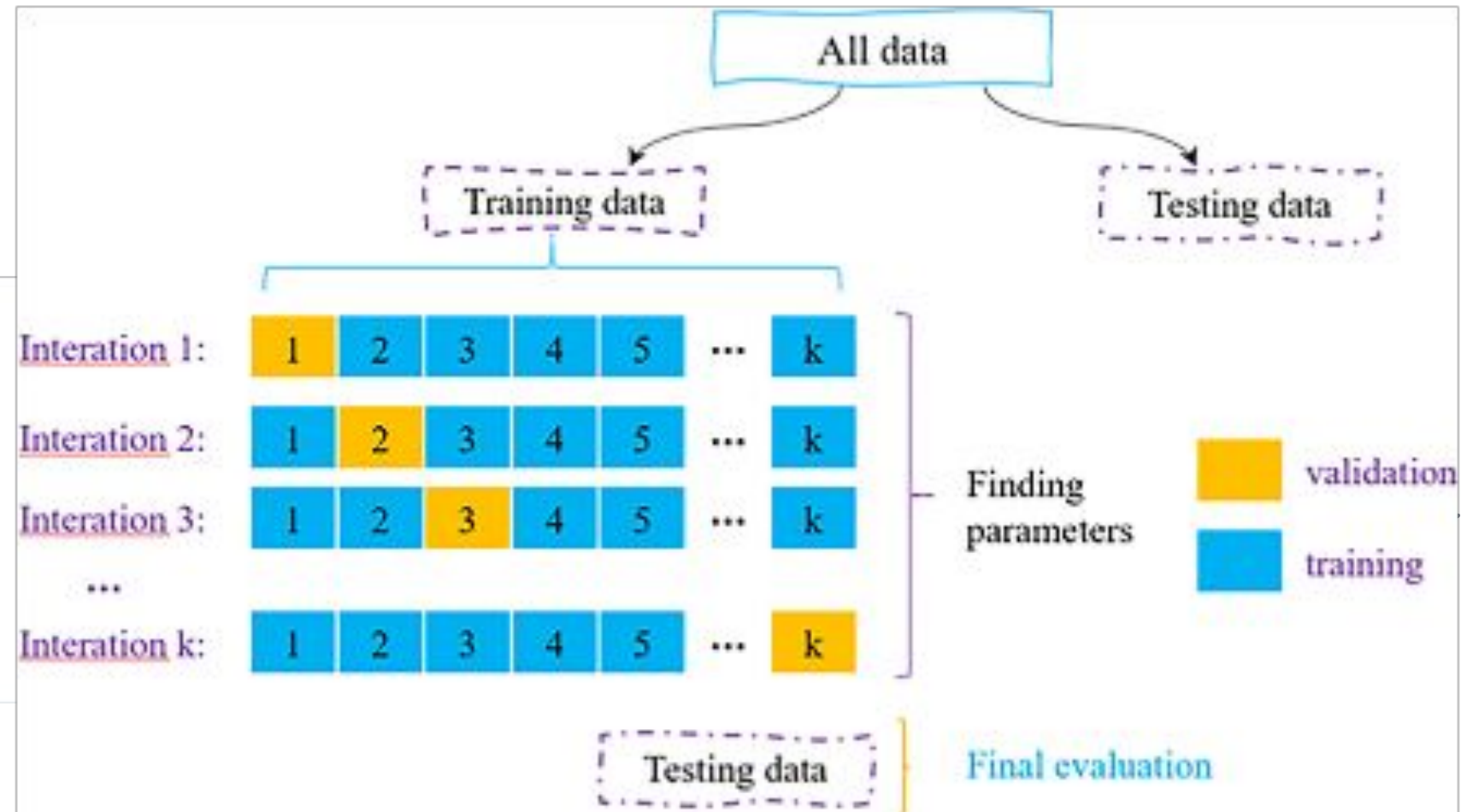
Initialize: 5 folds, shuffle data first

```
from sklearn.model_selection import KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
# Manual iteration over folds
for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    # Fit and evaluate your model here ...
```

Splits data into **K** equal parts (folds). Train on K-1, Test on

1.



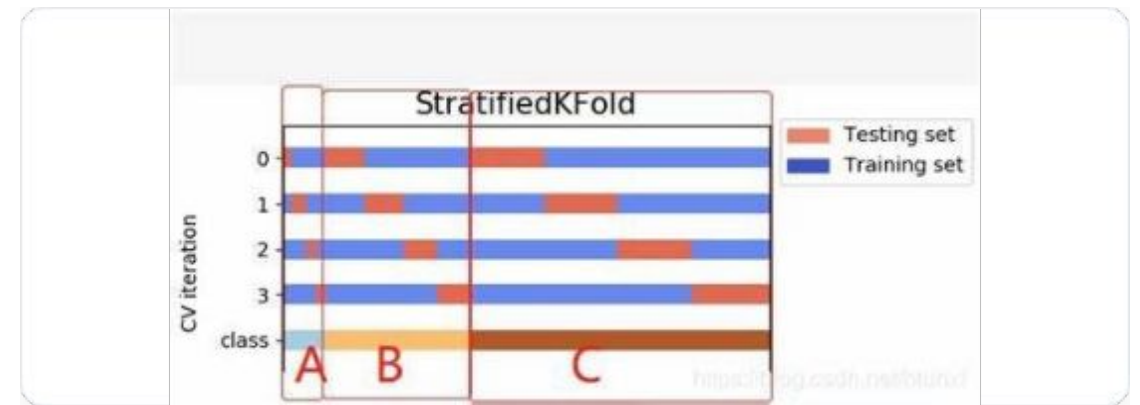
2. Stratified K-Fold

```
from sklearn.model_selection import
StratifiedKFold
skf = StratifiedKFold(n_splits=5)
# Notice we must pass 'y' to split() so it knows
the classes
for train_idx, test_idx in
skf.split(X, y): X_train, X_test = X[train_idx],
X[test_idx] y_train, y_test = y[train_idx],
y[test_idx] # Each fold now maintains class %
```

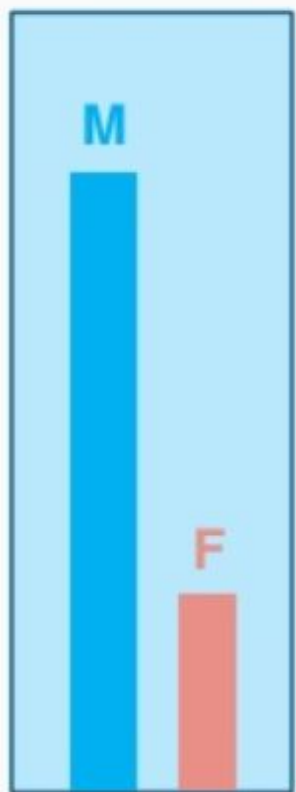
Why Stratify?

In classification, standard K-Fold might create a fold with **zero** examples of a minority class.

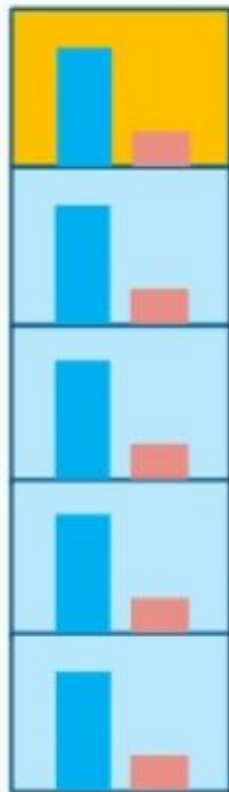
Stratification ensures each fold preserves the percentage of samples for each class found in the main dataset.



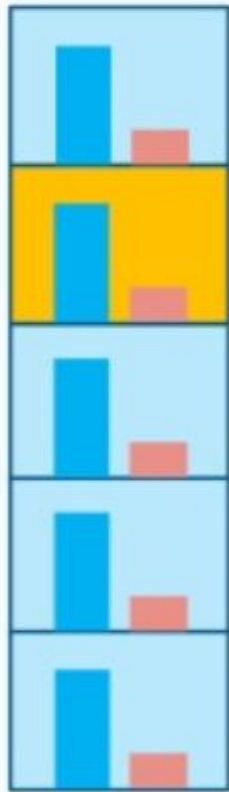
2. Stratified K-Fold



Class Distributions



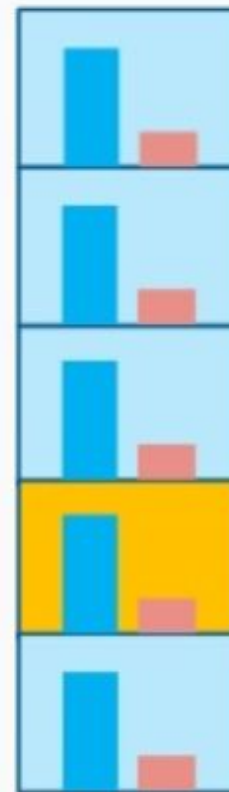
Round 1



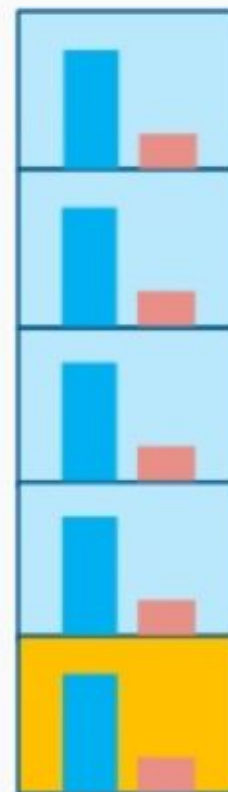
Round 2



Round 3



Round 4



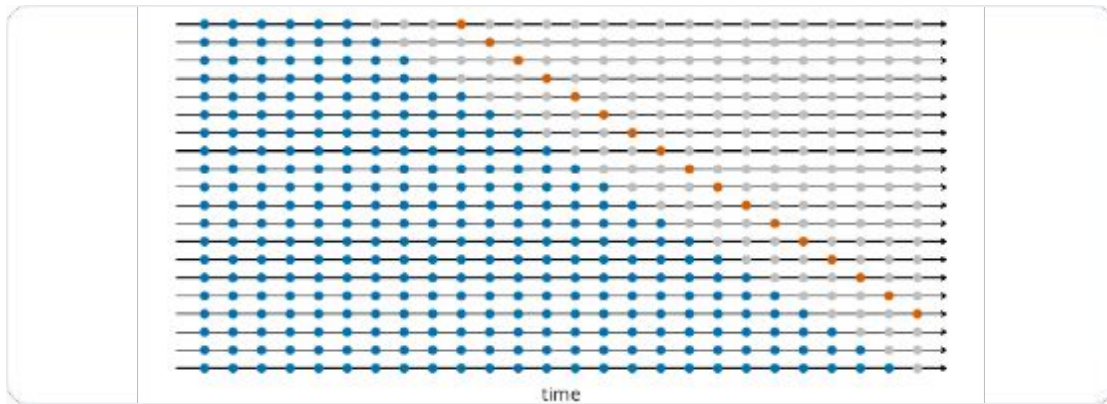
Round 5

3. Time Series Split

Respecting Time

We cannot shuffle time-series data. We must never train on the future to predict the past.

Rolling Origin: Successively increase the training set size and forecast the next chunk.



```
from sklearn.model_selection import
TimeSeriesSplit # Split into 5 temporal splits
tscv = TimeSeriesSplit(n_splits=5)
for train_index, test_index in tscv.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
```


	timestamp	open	high	low	close	volume
1	1569974460000	8325.0	8331.9	8325.0	8331.9	0.391844
2	1569974520000	8331.4	8331.4	8330.0	8330.0	0.142649
3	1569974580000	8324.9	8324.9	8324.9	8324.9	0.027575
4	1569974640000	8325.4	8328.3	8319.9	8328.3	52.710560
5	1569974700000	8330.0	8330.0	8323.3	8323.3	10.014685
6	1569974760000	8325.3	8325.3	8320.8	8320.8	0.041687
7	1569974820000	8320.8	8320.8	8317.3	8317.4	0.502247
8	1569974880000	8317.4	8317.4	8317.3	8317.4	0.114179
9	1569974940000	8312.3	8312.3	8312.3	8312.3	0.152000
10	1569975000000	8311.7	8311.7	8300.0	8300.1	1.697806

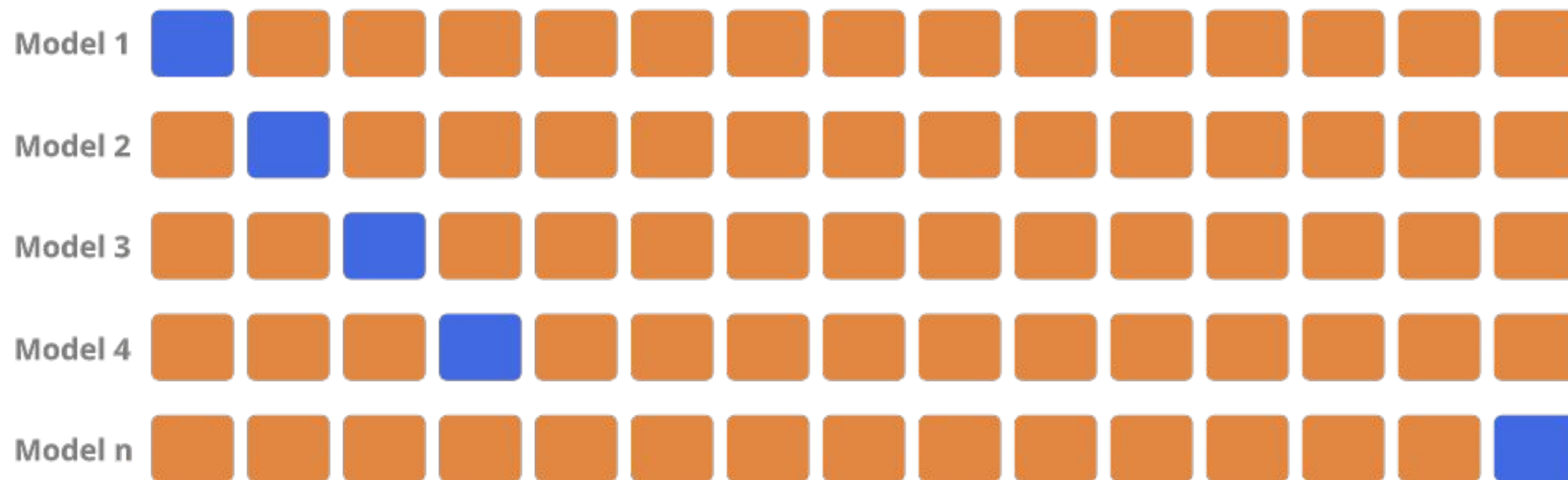
4. Leave-One-Out (LOOCV)

```
from sklearn.model_selection import LeaveOneOut
# Iterate over LOOCV (N = dataset size)
loo = LeaveOneOut()
for train_idx, test_idx in loo.split(X):
    # test_idx contains exactly 1 sample
    X_train, X_test = X[train_idx],
    X[test_idx]
    y_train, y_test = y[train_idx],
    y[test_idx]
```

When to use?

- **Small Datasets:** When you can't afford to hold out 20% of data for testing.
- **Pros:** Unbiased estimate (uses max training data).
- **Cons:** Expensive (N training runs) and high variance.

Leave-One-Out Cross Validation



Training
Data

Test
Data

The "Easy" Way

For standard evaluation, you rarely need to write the for loop yourself. Scikit-Learn provides a helper function that does it all in one line.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f"Scores: {scores}") print(f
    "Mean Accuracy: {scores.mean():.2f} (+/- {scores.std() * 2:.2f})")
```

Summary Checklist

- **Classification?** Use StratifiedKFold.
- **Regression?** Use KFold or RepeatedKFold.
- **Time Series?** Use TimeSeriesSplit.
- **Tiny Dataset?** Use LeaveOneOut.
- **Tuning Params?** Use GridSearchCV (it uses CV internally!).

```
# The Golden Rule  
if data_is_ordered:  
    use_time_series_split()  
else:  
    shuffle_and_stratify()
```